

Journal Pre-proofs

Full Length Article

Decision-making and confrontation in close-range air combat based on reinforcement learning

Mengchao YANG, Shengzhe SHAN, Weiwei ZHANG

PII: S1000-9361(25)00132-3
DOI: <https://doi.org/10.1016/j.cja.2025.103526>
Reference: CJA 103526

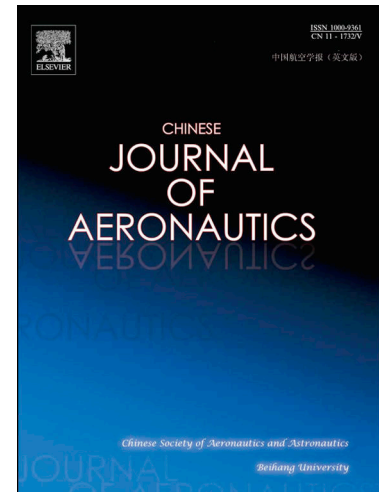
To appear in: *Chinese Journal of Aeronautics*

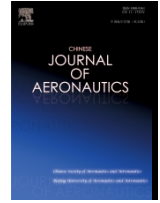
Received Date: 20 August 2024
Revised Date: 30 March 2025
Accepted Date: 31 March 2025

Please cite this article as: M. YANG, S. SHAN, W. ZHANG, Decision-making and confrontation in close-range air combat based on reinforcement learning, *Chinese Journal of Aeronautics* (2025), doi: <https://doi.org/10.1016/j.cja.2025.103526>

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier Ltd on behalf of Chinese Society of Aeronautics and Astronautics.



Contents lists available at [ScienceDirect](https://www.sciencedirect.com)**Chinese Journal of Aeronautics**Journal homepage: www.elsevier.com/locate/cja

Decision-making and confrontation in close-range air combat based on reinforcement learning

Mengchao YANG^a, Shengzhe SHAN^b, Weiwei ZHANG^{a, c, d,*}^a School of Aeronautics, Northwestern Polytechnical University, Xi'an, 710072, China^b 93995 Unit of the Chinese People's Liberation Army, Xi'an, 710072, China^c International Joint Institute of Artificial Intelligence on Fluid Mechanics, Northwestern Polytechnical University, Xi'an, 710072, China^d National Key Laboratory of Aircraft Configuration Design, Xi'an, 710072, China

Received 21 August 2024; revised 11 September 2024; accepted 3 January 2025

Abstract

The high maneuverability of modern fighters in close air combat imposes significant cognitive demands on pilots, making rapid, accurate decision-making challenging. While reinforcement learning (RL) has shown promise in this domain, the existing methods often lack strategic depth and generalization in complex, high-dimensional environments. To address these limitations, this paper proposes an optimized self-play method enhanced by advancements in fighter modeling, neural network design, and algorithmic frameworks. This study employs a six-degree-of-freedom (6-DOF) F-16 fighter model based on open-source aerodynamic data, featuring airborne equipment and a realistic visual simulation platform, unlike traditional 3-DOF models. To capture temporal dynamics, Long Short-Term Memory (LSTM) layers are integrated into the neural network, complemented by delayed input stacking. The RL environment incorporates expert strategies, curiosity-driven rewards, and curriculum learning to improve adaptability and strategic decision-making. Experimental results demonstrate that the proposed approach achieves a winning rate exceeding 90% against classical single-agent methods. Additionally, through enhanced 3D visual platforms, we conducted human-agent confrontation experiments, where the agent attained an average winning rate of over 75%. The agent's maneuver trajectories closely align with human pilot strategies, showcasing its potential in

decision-making and pilot training applications. This study highlights the effectiveness of integrating advanced modeling and self-play techniques in developing robust air combat decision-making systems.

Keywords: Reinforcement Learning; Decision Making; Air combat; Flight Simulation; Self-play

*Corresponding author. E-mail address: aeroelastic@nwpu.edu.cn

1. Introduction¹

Breakthroughs in artificial intelligence technology recently have had a profound impact on modern warfare. The upgrading of fighter aircraft and airborne equipment has made modern battlefield more complex. Integrated collaborative operations and escalating system complexity are placing new demands on intelligent battlefield awareness and maneuver decision-making¹⁻³. The U.S. Air Force began deploying military intelligence in the 20th century, and the Air Combat Evolution (ACE) project launched in 2019 has further accelerated the intelligent development of next-generation fighters⁴. China has also explicitly stated its intent to accelerate military modernization and the development of unmanned intelligent combat capabilities. The era of Intelligence has arrived⁵.

Modern air combat is dominated by manned fighters and unmanned aerial vehicles (UAV), categorized mainly into beyond-visual-range and close-range based on the combat distance between opposing forces⁶. With ongoing advancements in radar technology, airborne weaponry, and UAV systems, UAV swarm operations beyond visual range are emerging as a novel form of air combat. However, the prevalence of stealth technology and strategic maneuvers complicates efforts for fighter aircraft to engage enemies beyond visual range, often necessitating close-range air combat scenarios. Therefore, high mobility and flexible decision-making remain essential technologies for future air combat engagements⁷.

Common intelligent air combat decision-making methods can be divided into three types: differential games, expert systems and deep reinforcement learning (DRL)⁸.

The differential game method typically simplifies air combat scenarios into pursuit and evasion models, employing dynamic programming and game matrices to derive the Nash equilibrium. Isaacs⁹ of the RAND Corporation in the United States was the first to conduct comprehensive research on pursuit and evasion problems, systematically developing the theory of differential game methods and proposing numerous application scenarios. Baron¹⁰ expressed the differential equations in a linear quadratic form to solve the pursuit and evasion problems in one-on-one scenarios, highlighting differences between pilot and model control and demonstrating the differential game method's potential in control and combat applications. Park et al.¹¹ implemented maneuver generation using a scoring function matrix. Xu et al.¹² integrated situation awareness with the differential game method to facilitate semi-autonomous UAV maneuver decision-making. Gao et al.¹³ introduced a dynamic tactical control method based on game theory, applying it to a tripartite target-defender-attacker system. Due to its robust mathematical foundation, the differential game model offers the necessary interpretability for the field of air combat. However, constructing differential equations for target problems often demands numerous assumptions and environmental simplifications. Additionally, it depends heavily on the design of appropriate cost functions and restrictive conditions, rendering it impractical for real-world air combat scenarios.

The expert system method often employs

probabilistic models or fuzzy logic to select optimal actions from a repertoire of discrete maneuvers. Burgin et al.¹⁴ described maneuvers using multiple sets of discrete basic control quantities and employed adaptive maneuvering logic (AML) to achieve autonomous decision-making in a one-on-one air combat scenario. Akabari et al.¹⁵ applied fuzzy logic reasoning for offensive maneuver generation. Ernest et al.¹⁶ utilized genetic fuzzy systems to facilitate UAV decision-making in a high-fidelity simulation environment. İsci et al.¹⁷ used fuzzy logic to control the speed and attitude of UAVs, achieving superior results compared to the traditional PID algorithm. However, the development of an expert system introduces more subjective factors, and discrete maneuver plans fail to meet the high dynamics and flexibility requirements of actual battlefield conditions.

Currently, DRL is the predominant method in decision-making research, as it combines deep neural networks with RL to significantly enhance the model's nonlinear fitting capabilities. The advent of AlphaGo in 2016, which defeated the world champion, demonstrated the advantages of this method in decision-making domains¹⁸. Similarly, AlphaStar outperformed 99.8% of players in StarCraft¹⁹. Kaufmann et al.²⁰ applied RL to drone racing, achieving victory over the world champion. DARPA's application of DRL to air combat decision making resulted in the defeat of human pilots with a decisive advantage²¹. Consequently, the application of DRL methods in air combat decision-making has become a research hotspot, with numerous studies making substantial progress in this area.

According to the action space, DRL models can be categorized into two types: continuous and discrete. Discrete action methods typically utilize expert maneuvers as model outputs. Liu et al.²² applied the Deep Q-Learning (DQN) algorithm to UAV decision making under five-dimensional discrete action control, demonstrating the effectiveness of DQN in an air combat environment. Ma et al.²³ compared the DQN method with Minimax and Monte Carlo Tree Search (MCTS) methods, highlighting the long-term perspective of RL in decision-making problems. Yang et al.²⁴ selected a 7-dimensional discrete action library of classic maneuvers and incorporated curriculum learning to achieve enemy neutralization. Hu et al.²⁵ expanded the action library and used a Long Short-Term Memory (LSTM) network to verify the method's effectiveness in beyond-horizon decision-making scenarios. However, limited-dimensional discrete action outputs have certain shortcomings in practical applications.

With the advancement of RL algorithms, many scholars have applied continuous action space RL methods to air combat decision-making problems. Kurniawan et al.²⁶ researched and verified the use of the Actor-Critic architecture for air combat maneuver generation. Zhang et al.²⁷ proposed a final reward estimate based on Proximal Policy Optimization (PPO) and self-play (SP) algorithms to replace the optimization objective, thereby improving model decision-making performance. Wang et al.²⁸ used the heuristic Deep Deterministic Policy Gradient (DDPG) algorithm to enhance strategy exploration in continuous action scenarios. Sun et al.²⁹ employed hierarchical strategy gradient and SP methods to facilitate maneuver selection in attack and defense scenarios. Li et al.³⁰ achieved high winning rate decision-making under the 3-DOF dynamic model through PPO and Minimax algorithm training. Li et al.³¹ improved the training efficiency of the UAV decision-making model through parallel SP using the Soft Actor-Critic (SAC) algorithm. The neural network architecture with a self-attention mechanism was used to address the decision-making problem of UAV swarms in a discrete action environment³². Shan et al.^{33,34} built a training environment based on the Actor-Critic architecture and SP algorithm, providing a relatively complete intelligent air combat decision-making solution.

However, most research focuses on intelligent decision-making control using simplified 3-DOF dynamic models, which overlook the effects of the angle of attack and sideslip angle. While this simplification reduces the complexity of model training, it also diminishes the model's practical significance and application value. Additionally, much of the existing work remains at the theoretical verification stage, evaluating the feasibility of methods by analyzing trajectory curves in specific, isolated scenarios. These models often lack practical application validation and comprehensive performance evaluation. Consequently, decision-making models commonly suffer from poor generalization and limited applicability across diverse scenarios.

In view of the above problems, the main contributions of this paper are as follows.

- (1) Unlike most existing studies that utilize a simplified 3-DOF model, we employed high-fidelity open-source aerodynamic data^{35,36} to develop a 6-DOF dynamic model. In addition to the aerodynamic model, we incorporated the modeling and control of simplified radar and missile systems.

- (2) We incorporated pilot control experience and real fighter flight envelope performance to design a comprehensive RL environment. We proposed an optimized SP method that integrates expert strategies and combined it with an LSTM network³⁷, significantly enhancing decision-making and generalization capabilities.
- (3) Based on Unity³⁸, we developed a highly realistic virtual flight platform and conducted human-agent confrontation experiments with students and pilots. Finally, we evaluated the model's capabilities and application value from multiple perspectives, including maneuver trajectory analysis and model winning rate assessments.

2. Background and preliminaries

2.1. Basic air combat concepts

To express the situational relationships of fighters, assess the rationality of maneuvers, and design intelligent decision-making methods, it is essential to understand the basic terminology, geometric

relationships, and basic fighter maneuvers (BFM) of air combat.

Throughout this article, red represents friendly aircraft, and blue represents enemy aircraft. The geometric relationship and situational assessment of two aircraft on the battlefield mainly rely on distance, antenna train angle, and azimuth angle, respectively represented by R , χ and φ . These basic quantities are defined as follows. In the formulas, $\mathbf{R}$ and $\mathbf{V}$ represent the distance and velocity vectors under the northeast-down (NED) coordinate system, with the subscripts r and b indicating red and blue, respectively, as illustrated in Fig. 1.

$$\mathbf{R} = [x_b - x_r, y_b - y_r, z_b - z_r] \quad (1)$$

$$\chi = \arccos\left(\frac{\mathbf{V}_r \cdot \mathbf{R}}{\|\mathbf{V}_r\| \cdot \|\mathbf{R}\|}\right), \chi \in [0, \pi] \quad (2)$$

$$\varphi = \arccos\left(\frac{\mathbf{V}_b \cdot \mathbf{R}}{\|\mathbf{V}_b\| \cdot \|\mathbf{R}\|}\right), \varphi \in [0, \pi] \quad (3)$$

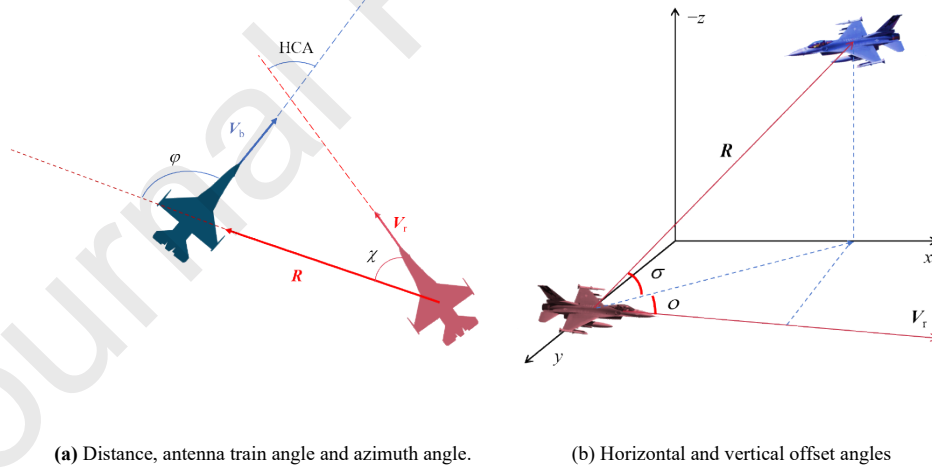


Fig. 1 Relative geometric relationship of fighters.

generally represents the pilot's line of sight when aiming at the enemy aircraft and can be approximated as the distance vector in this context.

In order to provide a clear description of the battlefield situation in three-dimensional space, we also introduce the horizontal and vertical offset angles, as well as the line of sight rotation angular velocity, denoted as θ , σ and LOS_ω , respectively. LOS

$$\theta = \arctan\left(\mathbf{R}_y^b / \mathbf{R}_x^b\right) \quad (4)$$

$$\sigma = \arcsin(\mathbf{R}_z^b / \|\mathbf{R}\|) \quad (5)$$

$$\text{LOS}_\omega = \|\mathbf{R}^w \times \mathbf{V}_{\text{delta}}^w\| / \|\mathbf{R}\|^2 \quad (6)$$

The air combat process is described by the observe-orientate-decide-act (OODA) loop³⁹. This loop involves making decisions based on the fighter's basic state variables and the aforementioned geometric quantities to control maneuvers, gain an advantageous position, and ultimately defeat the enemy. Consequently, these quantities are crucial for environmental observation and reward design in the following sections.

2.2. Reinforcement learning & self-play

RL is a fundamental machine learning paradigm used to solve sequential decision-making problems. The core principle involves an agent interacting with its environment and optimizing its policy based on feedback to maximize cumulative rewards⁴⁰. This process is typically modeled as a Markov decision process (MDP), which includes essential components such as state $\mathbf{S}$, action $\mathbf{A}$, reward R , discount factor γ and state transition probability P . The agent performs actions based on the current state, updates the battlefield situation according to state transition probabilities, and continuously refines its strategy through reward feedback to maximize the return. These elements and processes are illustrated in Fig. 2.

$$G_t = R_{t+1} + \gamma R_{t+2} + K = \sum_{T=0}^{\infty} \gamma^T R_{t+T+1} \quad (7)$$

$$\pi^* = \arg \max_{\pi} G_t \quad (8)$$

According to the method of solving the optimal strategy in Eq.(8), RL methods can be categorized into two main types: Value-Based and Policy-Based. Value-Based methods focus on value functions Eq.(9) and Eq.(10), using techniques such as dynamic programming, Monte Carlo, and temporal difference methods to perform value iteration. Actions are then selected based on these optimal values. Typical algorithms in this category include SARSA and Q-Learning⁴¹. In contrast, Policy-Based methods utilize the policy gradient theorem Eq.(11), treating the policy as the optimization target and directly updating its parameters.

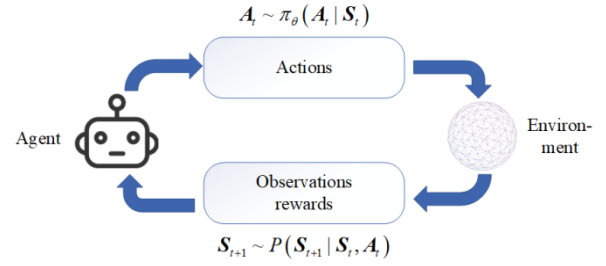


Fig. 2 General process of reinforcement learning.

Ψ_t represents the advantage function, which quantifies the benefit of taking a specific action a_t , compared to the expected value of the policy's default action at the same state. Notable examples of this approach are Proximal Policy Optimization (PPO)⁴², Deep Deterministic Policy Gradient (DDPG)⁴³, and Soft Actor-Critic (SAC)⁴⁴. The Actor-Critic (AC) architecture, proposed by Sutton, integrates the strengths of both Value-Based and Policy-Based methods. It reduces variance and facilitates the output of continuous actions⁴⁵, demonstrating excellent scalability and versatility in conjunction with various fundamental algorithms.

$$V^\pi(s) = E_\pi[G_t | \mathbf{S}_t = s] \quad (9)$$

$$Q^\pi(s, a) = E_\pi[G_t | \mathbf{S}_t = s, \mathbf{A}_t = a] \quad (10)$$

$$\nabla_\theta J(\pi_\theta) = E_\pi \left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \ln \pi_\theta(\mathbf{A}_t | \mathbf{S}_t) \right] \quad (11)$$

Establishing an opponent model based on rules in confrontation games remains a challenging task. Traditional single-agent RL methods use a fixed expert strategy for the opponent, which limits flexibility and can lead to model overfitting. Hernandez et al.⁴⁶ proposed a generalized RL SP approach that uses historical strategies as opponents, enhancing adaptability in adversarial environments⁴⁷. In SP, the opponent's strategy is also controlled by the model, with methods categorized into Naive SP, Fictitious SP, and δ -Uniform SP.

Building on these ideas, the algorithm proposed in this paper combines elements from these methods and uses SP and AC as its core structure, which will be detailed in Section 4.

3. Air combat platform construction

This section will outline the overall modeling of the intelligent air combat platform, covering the fighter model, airborne equipment models, and the construction of the visual platform.

3.1. Fighter model

To accurately reflect the aircraft's motion and meet practical requirements, we use 6-DOF nonlinear dynamic equations to establish the F-16 fighter model. Based on theoretical mechanics derivations, the dynamic equations in the aircraft body coordinate system are expressed in Eq.(12) and Eq.(13). The kinematic equations are then derived using the transformation matrix that converts coordinates from the body axes to the NED axes.

$$\begin{cases} \dot{x} = F_x / m - (wq - vr) \\ \dot{y} = F_y / m - (ur - wp) \\ \dot{z} = F_z / m - (vp - uq) \end{cases} \quad (12)$$

$$\begin{cases} \dot{p} = ((I_y - I_z)qr + I_{xz}(\dot{\alpha} + pq) + L) / I_x \\ \dot{q} = ((I_z - I_x)pr + I_{xz}(r^2 - p^2) + M) / I_y \\ \dot{r} = ((I_x - I_y)pq + I_{xz}(\dot{\beta} + qr) + N) / I_z \end{cases} \quad (13)$$

$$[\dot{\alpha}, \dot{\beta}, \dot{\gamma}]^T = T_b^e [u, v, w]^T \quad (14)$$

$$\begin{cases} \phi = p + (q \sin \phi + r \cos \phi) \tan \theta \\ \theta = q \cos \phi - r \sin \phi \\ \psi = (q \sin \phi + r \cos \phi) / \cos \theta \end{cases} \quad (15)$$

In Eqs.(12)-(15), $[x, y, z]$, $[u, v, w]$, $[p, q, r]$, and $[\phi, \theta, \psi]$ represent the basic state quantities of the aircraft: the position in the NED axes, velocity and angular velocity in the body axes, and Euler angles, respectively. I represents the inertia tensor, while L , M and N represent the roll, pitch and yaw moments respectively. Airspeed, angle of attack (α), and sideslip angle (β) can be derived from these quantities. Using the forces and moments exerted on the aircraft, the state change rates can be calculated with the aforementioned equations. Aerodynamic data interpolation tables, based on F-16 wind tunnel test data from NASA Langley Research

Center, are used to provide the forces and moments experienced during flight.

$$\begin{cases} L = C_{l_t}(\alpha, \beta, p, r, \delta_e, \delta_r, \delta_a) \bar{q} S b \\ M = C_{m_t}(\alpha, \beta, q, r, \delta_e) \bar{q} S \bar{c} \\ N = C_{n_t}(\alpha, \beta, p, r, \delta_e, \delta_r, \delta_a) \bar{q} S b \\ F_x = T + \bar{q} S C_{x_t}(\alpha, \beta, q, \delta_e) - mg \sin \theta \\ F_y = \bar{q} S C_{y_t}(\alpha, \beta, p, r, \delta_r, \delta_a) + mg \cos \theta \sin \phi \\ F_z = \bar{q} S C_{z_t}(\alpha, \beta, q, \delta_e) + mg \cos \theta \cos \phi \end{cases} \quad (16)$$

The coefficients in the above formulas include only the primary relevant quantities, with δ_e , δ_a and δ_r representing the deflections of the elevator, aileron, and rudder, respectively. Joystick and pedal commands adjust the throttle and control surfaces. By solving the force and dynamic equations and updating the aircraft state, the simulation cycle is completed, as illustrated in Fig. 3.

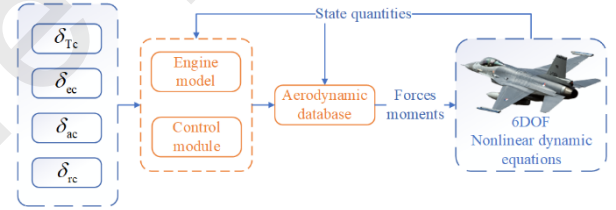


Fig. 3 Six degrees of freedom fighter simulation model.

In a test scenario involving stable horizontal flight at an altitude of 10 000 ft and a speed of 500 ft/s, an additional -2° deflection was applied to the elevator, and the flight simulation was conducted for 300 seconds. The results matched those of the open source simulation program⁴⁸, validating the aerodynamic model. As shown in Fig. 4, the red solid line represents the simulation results from the open-source program, while the dashed line represents the simulation results from our earlier implementation in C#. In our early research, we used a pure dynamics-based fighter aircraft model as the control subject. Due to the poor stability and control characteristics of the aircraft, the training results were unsatisfactory. After further adding damping and stability augmentation loops to the model, its performance improved to some extent. While this paper primarily focuses on developing a decision-making model that maps battlefield observations to joystick commands, with less emphasis on inner-loop control laws. Therefore, JSBSim, a widely used flight simulation software, was selected for secondary development in this study. A basic flight control system

(FCS) was implemented in JSBSim to meet the requirements of the controlled object. Dynamic-link libraries were generated to provide function interfaces for model selection, initialization, simulation progression, and state retrieval, and these were embedded into the intelligent air combat platform.

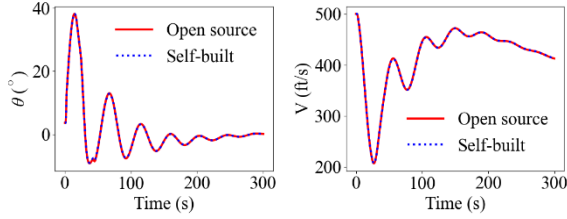


Fig. 4 Simulation test of $H=10\,000$ ft, $V=500$ ft/s, $\Delta\delta_e = -2^\circ$

3.2. Airborne equipment models

Relying solely on the position and attitude of the aircraft is insufficient to determine the outcome of an air battle. Therefore, we also modeled simplified systems for airborne radar, infrared missiles, and cannons. During close air combat, the aircraft primarily employs the radar's air combat maneuvering (ACM) mode, which includes three search sub-modes: horizontal search (ACM-HS), vertical search (ACM-VS), and fixed-axis search (ACM-BS), as shown in Table 1. Each mode differs in scanning area, time, and offset angle. When the enemy aircraft is detected within the scanning area and its speed is captured, the radar is deemed capable of intercepting the target. Once the radar successfully acquires the enemy, it transitions to single target tracking (STT) mode, which offers a tracking range of $\pm 60^\circ$ horizontally and $\pm 60^\circ$ vertically. The scan areas can be simplified to Fig. 5.

In addition to radar capabilities, we modeled four distinct launch modes for infrared air-to-air missiles, reflecting typical operational scenarios: fixed-axis aiming/fixed-axis launch (FAFL), fixed-axis aiming/off-axis launch (FAOL), fixed-axis scanning/off-axis launch (FSOL), and off-axis follow/off-axis launch (FFOL). The infrared seeker's scanning range exhibits variation across different missile launch modes, as summarized in Table 2. The FFOL mode, in particular, provides a more flexible and extensive search range, streamlining the interception of enemy aircraft and substantially reducing the pilot's workload. Consequently, the FFOL mode is utilized as the primary missile launch configuration.

Table 1 Radar scanning mode performance.

Mode	Horizontal range ($^\circ$)	Vertical range ($^\circ$)	Scan time (s)	Offset angle ($^\circ$)
HS	± 10	$[-15, 5]$	3	10
VS	± 5	$[-10, 30]$	3	5
BS	± 1	$[-1.5, 0.5]$	0.01	0

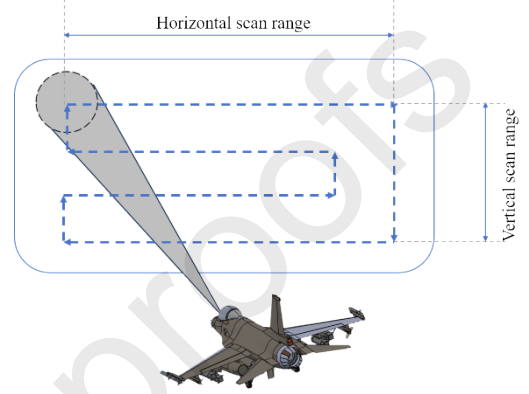


Fig. 5 Radar scanning area.

Table 2 Missile launch modes and characteristics.

mode	Infrared seeker scanning range
FAFL	Forward 2° cone angle
FAOL	Forward 2° cone angle
FSOL	Forward 5° cone angle
FFOL	Radar antenna direction 15° cone angle

The detection range of the missile's infrared seeker is expressed as a function of altitude and φ . Meanwhile, the attack envelope is determined by a combination of factors including altitude, speed, angle of φ , χ and the enemy's overload.

$$\begin{aligned} R_{\text{detec}} &= f_1(\varphi, H) \\ R_{\text{missile}} &= f_2(\varphi, \chi, H, V_r, V_b) \end{aligned} \quad (17)$$

For example, at an altitude of $H = 6\,000$ m, with velocities of $Ma_r = 0.8$ and $Ma_b = 0.6$, a load factor of $n_b = 1$, and $\chi = 0^\circ$, the detection range of the infrared seeker and the missile's attack envelope are illustrated in Fig. 6.

When the missile maintains a lock on the enemy for a specified duration, it is considered that the launch conditions are satisfied. The cannon, designed for rapid

firing at close range, has a maximum attack range set to 900 meters. Rapid fire conditions are met when the target is within this range and the angle between the tracking vector and our forward axis is less than 15° . The agent controls the radar scanning mode, offset direction, and seeker unlocking using the parameters $[r_m, r_p, m_u]$.

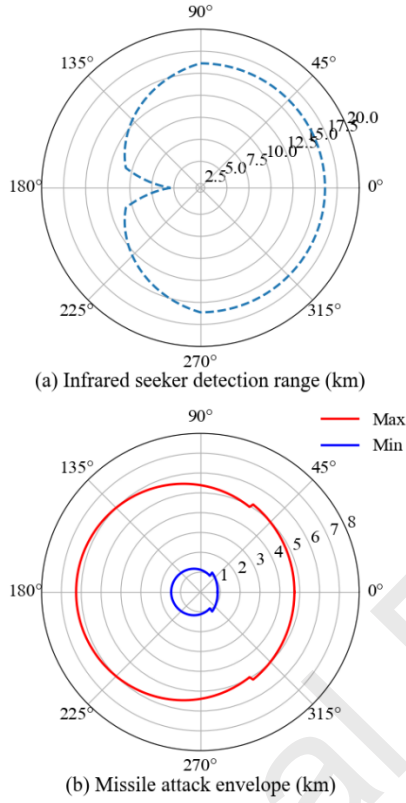


Fig. 6 Missile related range fitting curve with φ .

$$\text{Radar}_{\text{mode}} = \begin{cases} \text{HS} & r_m \in (1/3, 1) \\ \text{VS} & r_m \in (-1/3, 1/3) \\ \text{BS} & r_m \in (-1, -1/3) \end{cases} \quad (18)$$

$$\text{Radar}_{\text{offset}} = \begin{cases} \text{Left} & r_p \in (1/3, 1) \\ \text{None} & r_p \in (-1/3, 1/3) \\ \text{Right} & r_p \in (-1, -1/3) \end{cases} \quad (19)$$

$$\text{Missile}_{\text{seeker}} = \begin{cases} \text{Unlock} & m_u \in (1/2, 1) \\ \text{Lock} & m_u \in (-1, 1/2) \end{cases} \quad (20)$$

With the advent of modern fighter jets equipped with radar and off-boresight missile capabilities, air combat tactics have undergone significant changes. In the era of guns and non-off-boresight missiles, victory could only be achieved through prolonged tail-chase engagements. However, with modern advancements, success can now be secured through brief targeting windows. This paper aims to train strategies tailored to modern air combat, rather than those resembling the tactics of the Vietnam War era. From an application perspective, incorporating the control of weapon fire control systems and onboard equipment is essential for simulation training, as it provides a more realistic operational experience. This also supports further research into expanding the dynamic processes of weapon systems.

3.3. Visual system construction

A visual system was developed using Unity3D, which includes the rendering of battlefield terrain, fighter aircraft models, flight trajectories, detection ranges, explosion effects, and multi-perspective views. To facilitate the analysis of simulated trajectories from the decision-making model and to address the pilot's simulation needs, the visual effects of the flight simulation were enhanced. Additionally, features for flight data storage and playback were implemented.

To integrate hardware control, we employed Thrustmaster's Hotas Warthog stick and TFPR rudder, along with TrackIR equipment. The input interface was configured to support hardware-driven control within this platform. As illustrated in Fig. 7, this system will be utilized for trajectory analysis and human-agent confrontation testing in the subsequent sections.

3.4. Platform composition.

The platform constructed in this study is composed of three main components: the dynamics simulation module, the visual rendering module, and the agent training module. These modules are implemented using C++, C#, and Python, respectively. They are integrated into Unity through data communication and dynamic link libraries (DLLs), forming a unified framework. The overall architecture is illustrated in Fig. 8.

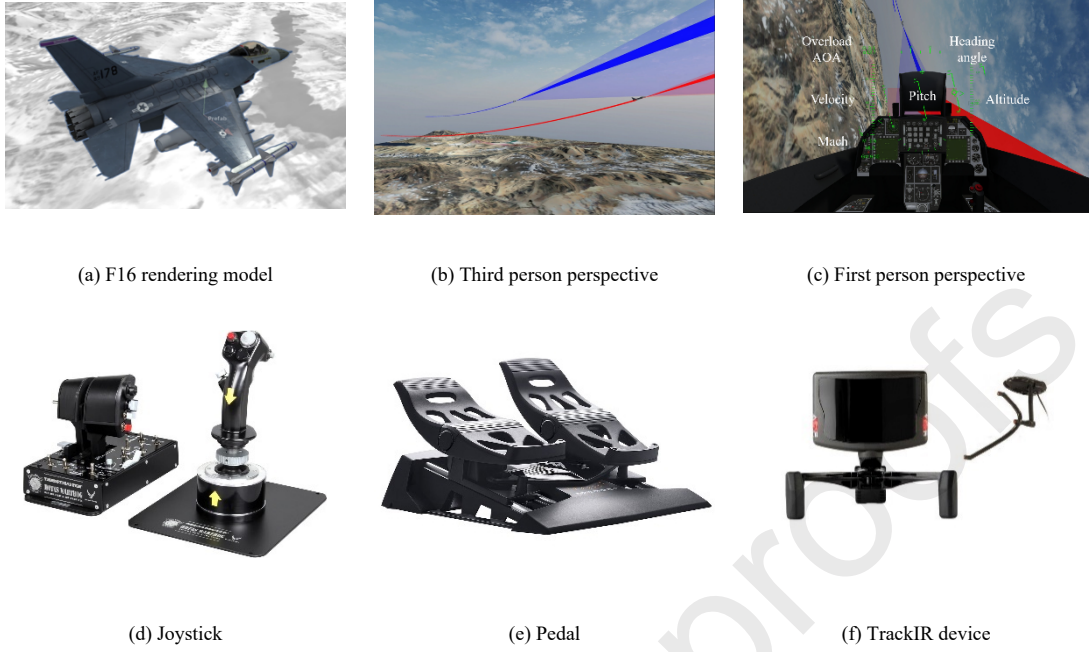


Fig. 7 Visual system and hardware equipment.

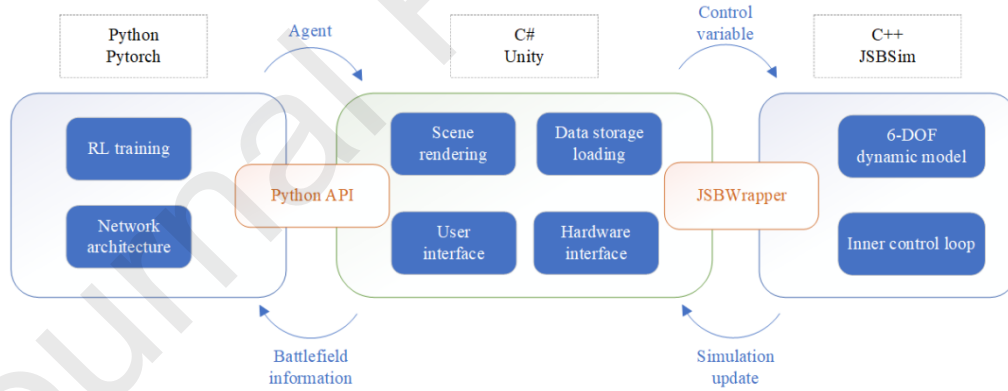


Fig. 8 Platform architecture.

When the platform starts, the scene is first initialized and rendered in Unity. Using the method interfaces provided by Unity MLAgents, the neural network model is initialized based on the specified hyperparameters, and the DLL functions are invoked to create and initialize the dynamics simulation model. Subsequently, at each simulation timestep, battlefield information is transmitted to the agent model via Python API data communication, where model inference is performed to obtain the actions, which are then sent back to Unity.

The control variables are passed to the dynamics module through DLL functions to advance the simulation, and the updated simulation state is rendered in Unity. Once the data volume meets the training batch requirements, the Python module updates the network model parameters according to the RL algorithm. This process repeats iteratively, achieving synchronized model training, simulation progression, and visual rendering.

4. RL environment design

4.1. Observation space

The fighter aircraft utilizes observation space quantities as input and generates action outputs through strategic model reasoning. Drawing from the human decision-making process, observation features are selected to accurately reflect the battlefield situation. In close air combat, pilots must maintain visual contact with the enemy, making it easier to obtain the basic status information of both parties. This paper extracts and normalizes features from battlefield information, encompassing the basic status, geometric status, action status, and weapon fire control status of both fighters. Specifically, the basic state variables consist of the aircraft's position, altitude, velocity magnitude, attitude angles, overload, and the states of the control surfaces and throttle. E denotes energy and e represents the quaternion. n_x , n_y and n_z represent the axial, lateral and normal overload, respectively. As the name implies, the geometric state variables represent the relative positional relationship between the two aircraft. $\mathcal{G}$ denotes the angle between the total external force and the distance vector. R , χ , φ , o , and σ have been detailed in the basic knowledge section. The weapon fire control state variables include the current radar and seeker horizontal and vertical scanning ranges, missile engagement range, cannon firing parameter, and command control variables.

Considering the timing characteristics of the decision-making process, we employ a time delay method for state space information stacking, incorporating the action and control surface states from the previous moment to smooth the output of the strategy model. Consequently, the tenth-order delay quantities of the 62-dimensional feature states listed in Table 3 are used as the actual input for the network model. These state variables generally reflect the battlefield situation and align with the information pilots require for decision-making. For simplicity, $\mathbf{O}_t$ in the following text refers to state observations that have undergone temporal delay stacking $\mathbf{O}_{\text{input}_t}$.

$$\begin{aligned} \mathbf{O}_t &= [x_t, y_t, H_t, e_t, \chi_t, \varphi_t, K] \\ \mathbf{O}_{\text{input}_t} &= [\mathbf{O}_{t-n}, \mathbf{O}_{t-n+1}, K, \mathbf{O}_t] \end{aligned} \quad (21)$$

Notably, to develop the most robust agent model possible, we provide the agent with a comprehensive set of information, excluding states that cannot be practically obtained, such as enemy weapon states and control surface states. Consequently, our input space has a higher dimension, and the weights of the various state variables are adjusted during the training process.

Table 3 Observation space characteristics.

Mode	Symbol	Dimension
Basic states	$[x, y, H]$	6
	$\ V\ $	2
	E^*	1
	e	8
	$[n_x, n_y, n_z]$	6
	$[\delta_a, \delta_c, \delta_r, \delta_T]^*$	4
	χ	2
	φ	2
	o	2
	σ	2
Geometric states	$\mathcal{G}$	2
	R	3
	$\ R\ $	1
	V_{delta}	3
	$\text{Radar}_{\text{hs}}^*$	2
	$\text{Radar}_{\text{vs}}^*$	2
	$\text{Seeker}_{\text{hs}}^*$	2
	$\text{Seeker}_{\text{vs}}^*$	2
	Missile_e^*	2
	Gun_a^*	1
Weapon fire control states	$[r_m, r_p, m_u]^*$	3
	δ_{Tc}^*	1
	$[\delta_{\text{ec}}, \delta_{\text{ac}}, \delta_{\text{rc}}]^*$	3

Note: * indicates that only own states are included.

4.2. Action space

On the premise of training convergence, we align with pilot control habits by using inputs from the steering column, pedals, and throttle lever as action quantities, and integrate these with the embedded control module to achieve end-to-end control of aircraft movement. Notably, the JSBSim model utilized in this paper incorporates a flight control module, meaning that action inputs are processed through the inner loop and do not directly translate to control surface positions. To accommodate adjustments to airborne equipment, we configure the environment to use a 7-dimensional continuous action space, expressed as

$$\mathbf{A} = [\delta_{\text{Tc}}, \delta_{\text{ec}}, \delta_{\text{ac}}, \delta_{\text{rc}}, r_{\text{mc}}, r_{\text{pc}}, m_{\text{uc}}] \quad (22)$$

$\delta_{Tc}, \delta_{ec}, \delta_{ac}, \delta_{rc}$ represent the throttle, elevator, aileron, and rudder commands, respectively. r_{mc}, r_{pc}, m_{uc} represent the radar scan mode, offset mode, and missile infrared seeker unlock command parameters, respectively. In conjunction with Eqs.(18)-(20), dynamic control is achieved through an inertial element.

4.3. Reward shaping

The goal of RL is to maximize cumulative reward, making the design of an effective reward function crucial for achieving intelligent decision-making. The most straightforward approach is to use result-based rewards; for instance, rewarding the agent when it successfully achieves the kill conditions. However, relying solely on such result-based rewards can lead to sparse feedback, which often hinders training convergence. To address this challenge, a more effective method is to use reward shaping. This involves guiding strategy training through dense process rewards, based on the air combat situation. By providing more frequent and informative reward signals throughout the decision-making process, reward shaping helps the agent learn optimal strategies more efficiently.

Based on the requirements of actual air combat scenarios, we reward actions that are advantageous to our position and penalize those that lead to disadvantages, resulting in a detailed and highly explanatory reward structure. It can be divided into situation guidance, event-triggered and result-based rewards. In the following reward design, λ represents the weights of the various rewards.

The close air combat situation is primarily evaluated using angle and energy metrics. Accordingly, the angle guidance reward is determined by the φ and χ , while the energy reward is assessed based on potential and kinetic energy, as detailed in Eq.(23) and Eq.(24). Being positioned behind the enemy's tail significantly increases our chances of a successful attack. This advantageous position not only hinders the enemy's maneuverability but also facilitates easier missile locking and launching conditions. Moreover, having superior energy during close air combat enables more rapid and agile maneuvers, further enhancing our tactical advantage. This synergy of positional and energetic superiority provides a decisive edge in engaging and overcoming adversaries.

$$\Phi_{\text{angle}} = \lambda_a \left(1 - \frac{AA + ATA}{180} \right) \quad (23)$$

$$\Phi_{\text{Energy}} = \lambda_e \left(\frac{E_r - E_b}{E_r + E_b} \right) \quad (24)$$

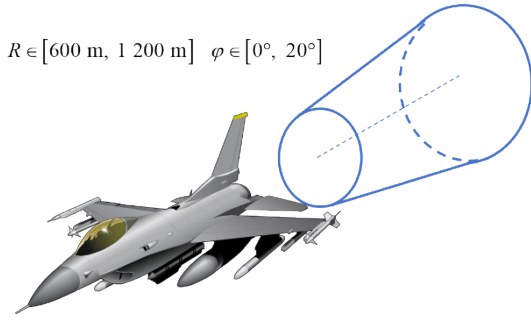
To further quantify and enhance our tactical positioning, we define the Offensive Control Zone (OCZ), a cone-shaped area behind the enemy aircraft, characterized by specific distance and angle parameters, as illustrated in Fig. 9(a). Being within this zone significantly improves our attack opportunities, as it complicates the enemy's ability to maneuver and evade. As indicated in Eq.(25), k_1 and k_2 are hyper parameters defining the zone's attributes, while $[a, b]$ and $[m, n]$ denote the boundary values for distance and angle. This formula can be intuitively represented as shown in Fig. 9(b), where the independent variables are the φ and the distance to the enemy. Points within this area correspond to the protruding region in the figure, where positive rewards will be given to the agent.

$$\begin{cases} \varphi_1 = 1 - \frac{1}{1 + \exp\left(\frac{\|R\| - a}{k_1}\right)} - \frac{1}{1 + \exp\left(\frac{b - \|R\|}{k_1}\right)} \\ \varphi_2 = 1 - \frac{1}{1 + \exp\left(\frac{\varphi - m}{k_2}\right)} - \frac{1}{1 + \exp\left(\frac{n - \varphi}{k_2}\right)} \\ \Phi_{\text{OCZ}} = 2\lambda_{\text{OCZ}}\varphi_1\varphi_2 \end{cases} \quad (25)$$

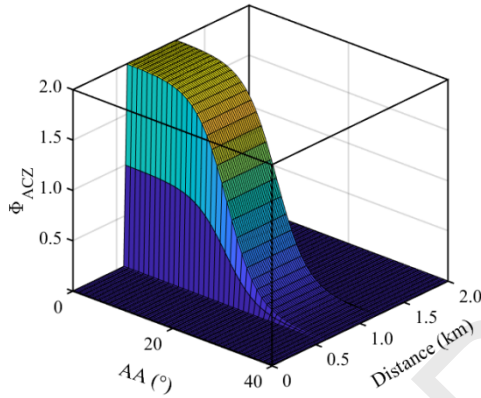
$$\Phi_R = \lambda_r \left(\left(\|R^*\| - \|R\| \right) / \|R^*\| \right) \quad (26)$$

$$\Phi_H = \lambda_h \left(\Delta H^* - |H_r - H_b| \right) / \Delta H^* \quad (27)$$

$$\Phi_T = \lambda_t \exp\left((T - T^*) / k_T \right) \quad (28)$$



(a) Schematic diagram



(b) Reward distribution

Fig. 9 Offensive control zone.

To avoid ineffective searches over excessively large areas and improve decision-making accuracy, additional guidance rewards are incorporated, focusing on engagement distance, altitude, and time.

When conditions for radar detection, locking, infrared seeker activation, missile targeting, and cannon attack are met, event-triggered rewards with assigned weights are applied (denoted by ν in the text), as shown in Table 4. To discourage the agent from focusing solely on accumulating these event rewards at the expense of overall outcomes, a penalty is introduced for the interruption of advantageous events. For example, if the agent deviates shortly after the missile lock is achieved, failing to meet the kill conditions and repeatedly collecting the locking reward, it demonstrates a misuse of the reward system. This behavior is penalized to ensure that the agent focuses on maintaining advantageous conditions rather than exploiting reward accumulation.

Table 4 Event-triggered rewards.

Event	Reward weight	Penalty weight
Radar detection	0.01	0
Radar lock	0.03	0
Seeker unlocking	0.03	0
Missile lock	0.05	-0.02
Meet missile kill conditions	0.06	-0.05
Meet gun kill conditions	0.05	-0.03

The result rewards are designed to reflect the outcomes of missile kills, gun kills, boundary violations, or collisions during the air combat scenario. Specifically, a missile kill is awarded if, after locking onto the enemy and maintaining them within the attack envelope for a specified duration τ_1 , the missile achieves its target. A gun kill is granted if the cannon attack is sustained for a specified duration τ_2 . Collisions are identified when the distance between the two aircraft falls below $d_{\min}$. Out-of-bounds situations are determined by both environmental constraints and fighter performance limits. Based on the F-16's flight envelope and aerodynamic data, constraints are imposed on maneuvering overload, Mach number, angle of attack, sideslip angle, and flight altitude, as presented in Table 5. Any breach of these constraints is deemed a failure.

Table 5 Boundary setting.

Restricted variable	Range	Weight
α (°)	[-20, 90]	0.08
β (°)	[-30, 30]	0.04
n_z	[-3, 9]	0.08
Ma	[0.1, 2]	0.08
Position (km)	[-0, 30]	0.08

$$f(x) = -1 + \frac{1}{1 + e^{k(x-b)}} - \frac{1}{1 + e^{k(x-a)}} \quad (29)$$

This paper designs a smoothing function with soft constraints to provide gradient information for boundary limits, improving model training by preventing abrupt changes in boundary rewards. In Eq.(29), a and b represent the lower and upper boundaries of the variable, respectively. For example, with the angle of attack α , the penalty is near zero when α is far from the boundary, but increases rapidly as α approaches it. This approach mitigates the issue of gradient disappearance and ensures more effective learning by avoiding sudden reward changes, as shown in Fig. 10.

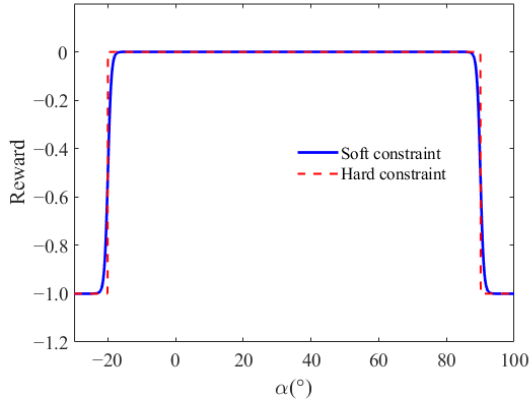


Fig. 10 Soft constraints on angle of attack.

The result reward, serving as the primary objective, carries the highest weight and is detailed in Table 6. The total reward feedback is computed as a weighted sum of all reward components. Given the confrontational nature of the environment, both guidance and event rewards must account for the states and actions of both the enemy and our own aircraft.

Table 6 Result rewards.

Result of confrontation	Weight
Missile kills enemy	1
Cannon kills enemy	0.9
Enemy is out of bounds	0.8
Killed by enemy missile	-1
Killed by enemy cannon	-0.9
Self out of bounds	-0.8
Draw game	0

$$\begin{cases} R_{\text{situation}} = \Phi_r - \Phi_b \\ R_{\text{event}} = v_r - v_b \\ R_{\text{total}} = R_{\text{situation}} + R_{\text{event}} + R_{\text{result}} \end{cases} \quad (30)$$

So far, the construction of the modeling and RL environment has been completed. The interaction process between the fighter and the battlefield can be described as follows: the agent obtains manipulation instructions A_t from the algorithm model based on the battlefield state O_{input} over a period of time. These

instructions are passed to the dynamic simulation module to update the battlefield state. The environment then provides reward feedback R_{total} to the agent based on the situation at each moment, generating trajectory samples in a loop for subsequent model training.

5. Method implementation

The construction of the model enhances the realism of the scene, and the design of the environment dictates the optimization direction of the strategy. To achieve a high level of decision-making, the optimal design of the algorithm is essential. In this section, we optimize existing algorithms from two perspectives: the training process and network architecture.

5.1. Optimized self-play algorithm

Self-play reinforcement learning is a method where agents train by competing against themselves or other agents, thereby enhancing their decision-making capabilities and strategy diversity. The theoretical basis of this approach lies in the continuous adaptation of the agent's strategy to thrive in a dynamic adversarial environment. By iteratively refining strategies, agents can overcome limitations and progressively enhance their overall performance. SP RL is commonly applied in multi-agent systems and game theory, such as in complex strategy games like Go and chess, as well as in practical applications like autonomous driving and drone control.

Although the SP method solves the problem of opponent modeling, the above traditional training methods will still lead to model degradation. That is to say, model A can achieve good results against model B. Model B can defeat model C, but model A will be defeated by C. In order to alleviate Red Queen Effect, we propose an optimized SP algorithm, which introduces expert maneuver into the opponent selection mechanism, randomly initializes training scenarios, effectively increases the diversity of historical model communities, and improves the generalization ability of the strategy. The overall architecture of the algorithm is shown in the Fig. 11.

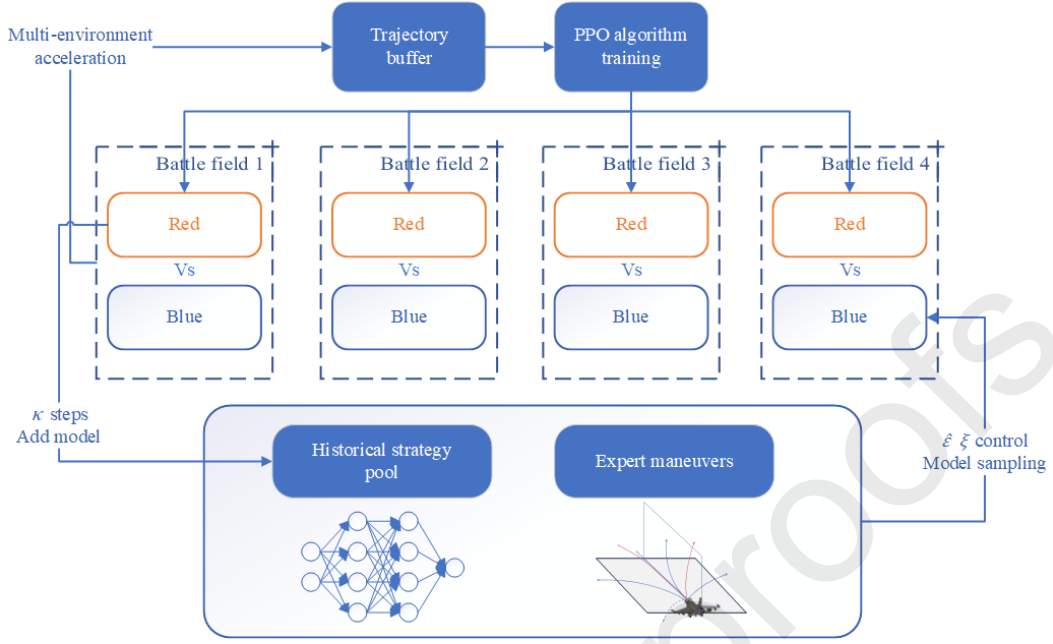
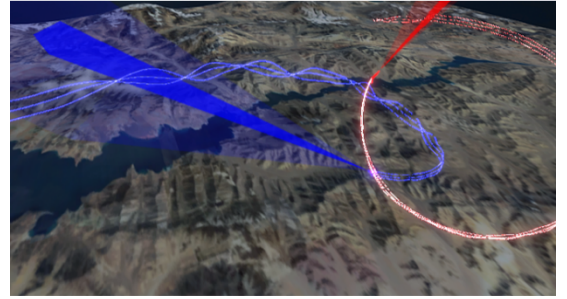


Fig. 11 Optimized self-play architecture.

The optimized SP algorithm features eight unique hyperparameters: $\langle M, \tau, \zeta, \kappa, \xi, \hat{\epsilon}, t_1, t_2 \rangle$. Every κ training steps, the current policy slice is added to the model pool. If the strategy pool exceeds M models, the earliest model is discarded. The opponent's strategy is reselected every τ steps, and the training team alternates after the interval ζ is met. Each time an opponent is selected, there is a probability ξ of selecting a fixed maneuver from the expert action library and a probability $1 - \xi$ of drawing from the strategy pool. This paper uses a library of seven fixed maneuvers, including horizontal flight, turning, descending, climbing, and inclined climbing. If drawn from the strategy pool, there is a probability $\hat{\epsilon}$ of selecting the latest model as the opponent and a probability $1 - \hat{\epsilon}$ of performing uniform sampling. This strategy extraction rule is inspired by the shortcomings of the team's previous work. We observed that while the single-agent algorithm and naive SP methods perform well against specific scenarios and opponents, they often exhibit passive behavior when faced with new scenarios or changing opponent strategies. As shown in Fig. 12(a), when the blue side maintains level flight, the agent fails to make effective decisions and opts for passive combat. To address this, we propose using historical models as opponents and incorporating basic actions, akin to mutation operations in genetic algorithms. This approach significantly enhances the diversity of opponents and improves the model's robustness.



(a) Schematic diagram



(b) Reward distribution

Fig. 12 Problems with basic model.

Additionally, we employ curriculum learning to gradually expand the random initialization range of the battlefield as training progresses. This method, inspired by human learning processes, starts with simple

scenarios to build basic capabilities when the model's proficiency is low and then progressively increases the complexity of the scenarios. Once the number of steps exceeds t_1 , random initialization of positions is introduced. After the number of steps reaches t_2 , random initialization of the yaw angle is incorporated, which is shown in Table 7. This approach helps to broaden the model's application scenarios while facilitating training convergence.

Table 7 Battlefield initialization.

Stage		$x(\text{km})$	$y(\text{km})$	$\psi(^{\circ})$	$V(\text{m/s})$
$t < t_1$	Red	-1	1	0	182.8
	Blue	1	-1	180	182.8
$t_1 \leq t \leq t_2$	Red	$[-4, 2]$	$[-3, 3]$	0	182.8
	Blue	$[-2, 4]$	$[-3, 3]$	180	182.8
$t_2 \leq t$	Red	$[-4, 2]$	$[-3, 3]$	$[0, 360]$	$[150, 250]$
	Blue	$[-2, 4]$	$[-3, 3]$	$[0, 360]$	$[150, 250]$

In order to improve training efficiency, we adopt a multi-process parallel acceleration method to create multiple battlefields and speed up trajectory generation. The basic flow of the algorithm is shown in Algorithm 1.

Algorithm 1 Optimized Self-Play

INITIALIZATION

Policy net θ , value net ω , strategy pool P , data buffer D and various hyperparameters

Create multiple battlefields according to settings

SAMPLING

while $t < t_{\max}$ **do**

if $\Delta t > \kappa$ **then**

$P \leftarrow P \cup m_{\theta}$

if $\Delta t > \zeta$ **then**

change training side

if $\Delta t > \tau$ **then**

if random number $\zeta_1 < \zeta$ **then**

select randomly $\pi_{\text{opp}} \in \text{Expert Maneuvers}$

else

if random number $\zeta_2 < \hat{\varepsilon}$ **then**

select $\pi_{\text{newest}} \in P$

else select randomly $\pi_{\text{opp}} \in P$

generate and save $D \leftarrow D \cup \langle \mathbf{O}_t, \mathbf{A}_t, R_{t+1}, \mathbf{O}_{t+1} \rangle$

if one game meets end conditions **then**

randomly reset the battlefield according to t

TRAINING

if $|D| > \text{Batchsize}$ **then**

update parameters using PPO

$\omega \leftarrow \omega_{\text{old}} - \alpha_{\omega} \nabla \text{loss}_{\omega}$

$\theta \leftarrow \theta_{\text{old}} - \alpha_{\theta} \nabla \text{loss}_{\theta}$

5.2. Network architecture

In the early research on RL, the focus was typically on environmental design and algorithmic logic. Many related studies used fully connected networks as reasoning models, in contrast to the detailed network architecture design seen in deep learning research. Considering the characteristics and challenges of DRL, this paper further optimizes the methodology from the perspective of network architecture.

In this paper, we introduce a LSTM layer into the agent model to address the limitations of traditional fully connected networks, offering several advantages particularly beneficial for RL tasks. LSTMs are designed to handle sequential data and maintain a memory of past events, allowing the model to learn from

temporal dependencies and make more informed decisions—a crucial aspect of RL, where the current state often depends on a sequence of previous actions and observations.

The LSTM network leverages historical information over time, aligning with human decision-making processes by considering the progression of prior states. This enhances the timing and continuity of the strategy model, ensuring decisions are contextually informed rather than isolated. With specialized gating mechanisms, LSTMs retain important information over long sequences while discarding irrelevant data, leading to more stable and efficient learning. Additionally, LSTMs improve the model's generalization across different scenarios and help mitigate overfitting.

The overall network model employs an Actor-Critic architecture, utilizing stacked observations—specifically, a 620-dimensional state space—as input to guide decision-making. In this setup, the Actor is responsible for generating actions that constitute the core of the strategy model, while the Critic provides value estimates crucial for optimizing the model through loss calculation. To enhance the model's ability to process sequential information and improve decision-making continuity, a single LSTM layer is integrated after feature extraction. The network is configured with eight hidden layers, each containing 512 neurons, to effectively capture complex features from the input data. The LSTM layer has a hidden size of 256 and processes sequences of length 64. This means that post feature extraction, the output is segmented into sequences of 64, which the LSTM processes to produce a 256-dimensional feature representation. Subsequently, the Actor maps this representation to a 7-dimensional continuous action space, while the Critic maps it to a single value corresponding to the evaluated state's value. The complete architecture is illustrated in Fig. 13.

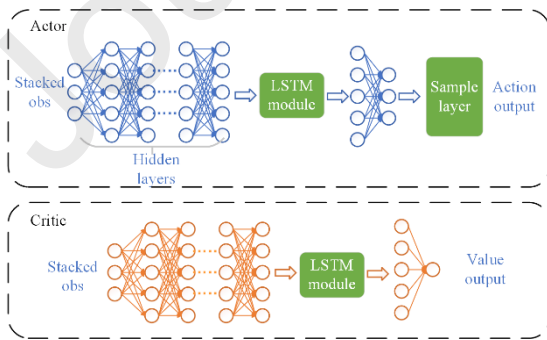


Fig. 13 LSTM actor-critic network architecture.

Balancing exploration and exploitation is a significant challenge in RL, especially when seeking a strategy that not only converges but also exhibits strong generalization across various scenarios. One common pitfall is that agents may settle into local optima, failing to explore alternative strategies or states that could lead to better performance. To address this issue, we introduced the Intrinsic Curiosity Module (ICM), a mechanism designed to encourage exploration by rewarding the agent for discovering novel states.

The ICM is based on the concept of intrinsic motivation, where an agent is driven not just by external rewards but also by internal signals that reflect the novelty or unpredictability of its environment. By integrating the ICM into our model, we aim to enhance the agent's ability to explore and learn from a wider range of experiences during training, thus improving its overall robustness and adaptability.

The ICM consists of two key components: the forward network and the inverse network, which is shown in Fig. 14. The inverse network takes as input the state transitions $\langle \mathbf{O}_t, \mathbf{O}_{t+1} \rangle$ and uses the action $\mathbf{A}_t$ as the label for training. This network's primary function is to perform feature extraction, isolating the elements of observed state changes that are directly influenced by the agent's actions, thereby filtering out irrelevant information.

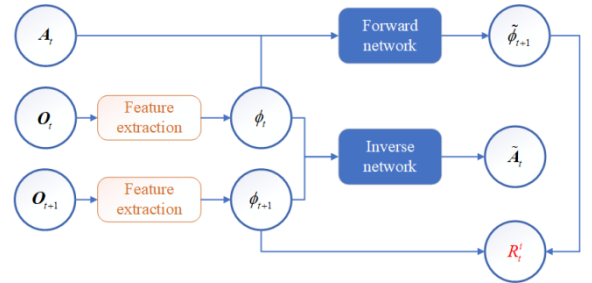


Fig. 14 Intrinsic curiosity module architecture.

The forward network, on the other hand, predicts the next state based on the current state $\mathbf{O}_t$ and action $\mathbf{A}_t$. The prediction is then compared to the actual next state, and the difference between the predicted and actual state representations—specifically, the difference between ϕ_{t+1} and ϕ_{t+1}^o is used to generate an intrinsic curiosity reward. This reward incentivizes the agent to explore states that are less predictable or previously unencountered, fostering a deeper understanding of the environment and reducing the likelihood of getting stuck

in repetitive or suboptimal behavior patterns.

It is important to note that the ICM provides these additional rewards only during the training process, enhancing the agent's exploration capabilities without altering the underlying structure of the policy model. Specifically, we utilized the ICM module built into ML-Agents, which employs a straightforward fully connected network architecture with three hidden layers, each containing 256 neurons. To prevent the intrinsic curiosity rewards from interfering with the strategy optimization direction, we set the reward weight for the ICM module to a modest value of 0.001 in our experiments. This ensures that while the module effectively encourages exploration, it does not disproportionately affect the agent's primary learning objectives.

6. Model training and configuration

The convergence of the basic single-agent algorithm in an air combat environment has been previously validated by our team's work³³. To emphasize the enhancements introduced by the algorithm proposed in this paper, we also present the training curve of the classical single-agent algorithm, PPO, as shown in Fig. 15. The reward curve demonstrates rapid convergence to a value of 1, while the episode length curve initially decreases from long to short. This indicates that after an exploratory phase, the agent successfully learns to execute optimal actions to eliminate the enemy efficiently.

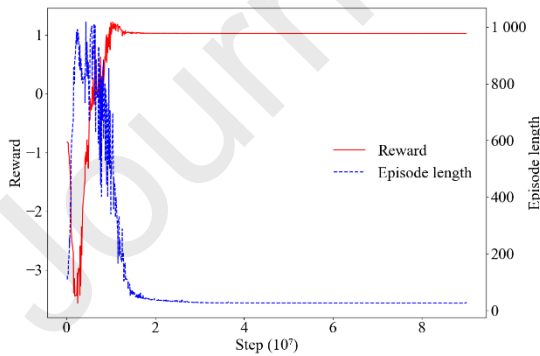


Fig. 15 Training curves of PPO algorithm.

In SP RL, evaluating an agent's performance using only single-agent reward metrics can be problematic. Traditional reward-based evaluations often emphasize short-term gains or the exploitation of specific opponent

weaknesses, which may not accurately reflect the agent's true strategic capabilities. This can result in a model that performs well in particular scenarios but lacks the generalization required to succeed across a diverse range of opponents and situations.

To address this shortcoming, we employ the ELO rating system, a widely recognized method for ranking competitors based on their relative performance in games. The ELO system allows us to dynamically monitor the model's progress by providing a numerical score that reflects its strength against a variety of opponents. As the diversity of opponent models increases during training, a higher ELO score becomes a key indicator of the model's evolving abilities and robustness.

$$EX = \frac{1}{1 + 10^{(ELO_{opp} - ELO_{self})/400}} \quad (31)$$

$$ELO_{self} \leftarrow ELO_{self} + K(U - EX)$$

After each air battle within the SP framework, the model's ELO score is updated using Eq.(31). Here, EX represents the expected win rate, which is calculated based on the current ELO ratings of the competing models, and U denotes the actual result of the confrontation. This update mechanism ensures that the ELO rating accurately reflects the model's performance, encouraging continuous improvement and providing a clear metric for evaluating progress throughout the training process. Unlike reward-based metrics, ELO ratings can reach higher values after training against opponents with varying levels of skill.

The training configuration is detailed in Table 8. We trained three approaches: the basic SP method, the method enhanced with the ICM and random initialization, and the complete method proposed in this paper. All methods demonstrated effective convergence, with their respective training curves shown in Fig. 16.

Table 8 Training configuration.

Object	Name	Value
FNN	input size	620
	layers	8
	hidden units	512
	memory size	512
LSTM	sequence length	64
	hidden dim	256
	layers	3
ICM	hidden units	256
	additional reward weight	1×10^{-3}

PPO	learning rate α	1×10^{-5}
	threshold ε	0.1
	buffer size	204800
	batch size	2048
	discount γ	0.99
	epoch	5
Self Play	M	100
	τ	2×10^3
	ζ	1×10^5
	κ	2×10^4
	ξ	0.05
	$\hat{\varepsilon}$	0.5
	t_1	5×10^6
	t_2	1×10^7
initial ELO		1200

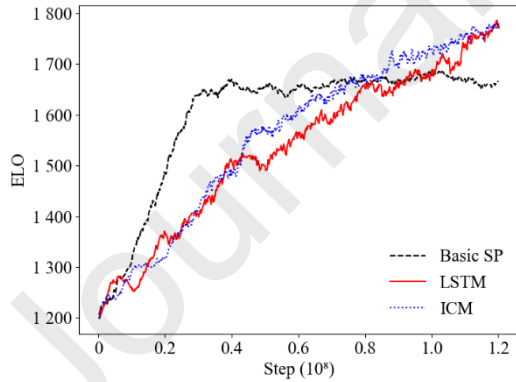


Fig. 16 ELO training curves for different methods.

The fully connected network serves as the basic network structure for both the PPO and basic SP methods. It takes a 62-dimensional state vector with a 10-step delay as input and consists of eight hidden layers with 512 neurons each. This network also forms the foundation for integrating the ICM and LSTM modules. We set the initial ELO rating to 1 200. This standard

starting point represents an average skill level, providing a baseline from which the model's performance is evaluated. The basic SP curve flattens around 3×10^7 training steps, indicating that the model has fallen into a local optimum. In contrast, the other two methods, which incorporate random initialization and exploration encouragement mechanisms, exhibit a slightly slower initial training rate. However, their curves rise steadily over time, suggesting a more robust and progressive learning process.

Although real-time training is challenging for reinforcement learning methods, converting the trained models to Open Neural Network Exchange (ONNX) format and selecting appropriate hardware for inference using C/C++ can support real-time decision-making^{49,50}. Testing results, as shown in Table 9, indicate that while the inclusion of LSTM layers and larger model sizes slightly increase inference time, the delay remains within acceptable limits for real-time applications.

Table 9 Model inference performance.

Model	Device	Time (ms)
SP-FNN	(Laptop) AMD Ryzen 7 5800H	1.8
SP-LSTM	(Laptop) AMD Ryzen 7 5800H	2.5
SP-FNN	Intel(R) Core(TM) i5-14600KF	1.5
SP-LSTM	Intel(R) Core(TM) i5-14600KF	1.7

7. Experiments and analysis

To evaluate and demonstrate the decision-making capabilities of the model, this section provides an analysis of decision trajectories across different scenarios, compares the win rates of various algorithmic models, and presents results from human-machine confrontation experiments.

7.1. Performance analysis

We established several scenarios as outlined in Table 10 to analyze the decision-making trajectories of the optimized SP algorithm model proposed in this paper, which incorporates LSTM and ICM modules.

Table 10 Initial status of different scenarios.

scene	airplane	model	x(km)	y(km)	$\psi(^{\circ})$
One	red	LSTM	-1	1	0
	blue	Heuristic	1	-1	180
Two	red	LSTM	-1	1	0
	blue	LSTM	1	-1	180

Three	red	LSTM	$[-3, 3]$	$[-3, 3]$	$[0, 360]$
	blue	LSTM	$[-3, 3]$	$[-3, 3]$	$[0, 360]$

Scenario 1 depicts a situation where two aircraft face each other, with the blue aircraft executing fixed maneuvers. As shown in Fig. 17(a), the red fighter successfully performs rapid turns, tracks the blue aircraft's tail, and achieves a missile kill. Scenario 2 involves both agents starting in a balanced head-on position. The resulting trajectory demonstrates a typical Two-Circle battle strategy, where both sides attempt to gain an angle advantage, turning toward the enemy and engaging in continuous combat. As energy is depleted, the trajectory exhibits a downward spiral, culminating in

one aircraft securing a kill. Scenario 3 introduces random initialization of position and yaw angle, testing the model's effectiveness across varied situations. Fig. 17(c) illustrates how the red aircraft, with an initial advantage, quickly turns toward the blue aircraft and completes the missile kill.

As previously mentioned, the single-agent algorithm and basic SP methods suffer from issues like overfitting, redundant actions, and irrational behavior when confronted with different scenarios and opponents, as shown in Fig. 12. In contrast, the method proposed in this paper effectively handles various scenarios and strategic opponents, enabling successful engagement and significantly enhancing the model's decision-making capabilities.

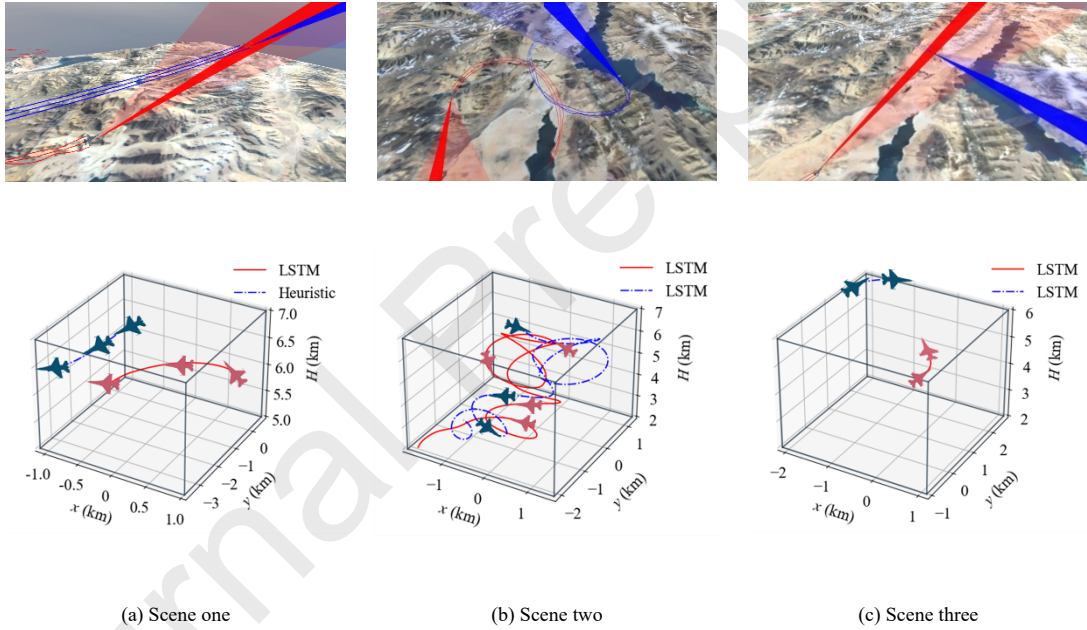


Fig. 17 Trajectories under various test scenarios.

7.2. Model confrontation

Furthermore, from an algorithmic perspective, models trained with different methods were tested in confrontation experiments. Using the typical head-on situation of close air combat as the initial state, the results were calculated over 300 engagements. In this context, LSTM represents the final optimized SP method, ICM refers to the model with the curiosity module and random initialization, BASE denotes the basic SP model, and PPO stands for the classic single-agent model.

The results clearly demonstrate that the optimized SP method significantly enhances the decision-making capabilities of the model, achieving a higher win rate against basic models. Fig. 18 illustrates the win rate from our camp's perspective, with darker colors indicating a higher success rate. The PPO model is only effective in fixed scenarios, exhibiting redundant actions and overfitting during confrontations, as evidenced by repeated boundary violations shown in Table 11. While the ICM model encourages exploration, leading to a more aggressive approach, it also increases the likelihood of irrational and out-of-bounds maneuvers. The final model, which incorporates the LSTM network architecture and expert maneuvers, addresses these

shortcomings, enhancing the rationality and continuity of decision-making.

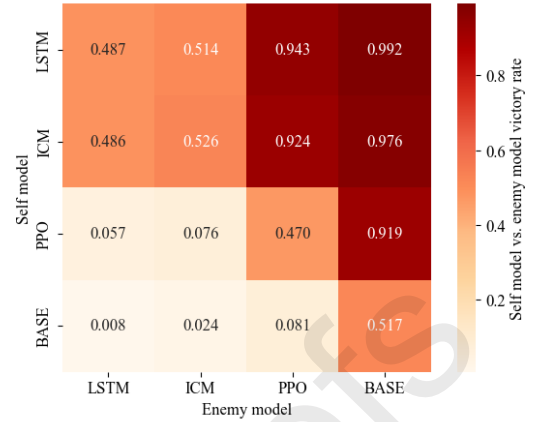


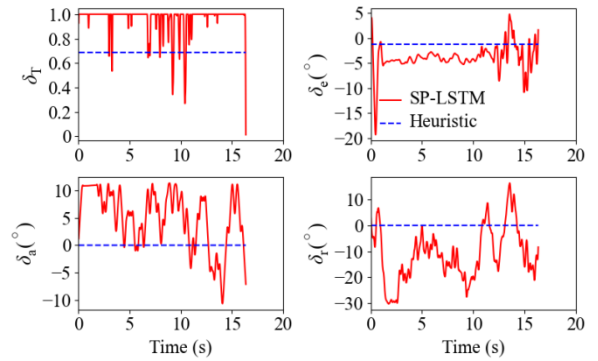
Fig. 18 Model confrontation winning rate heat map.

Table 11 Model confrontation result.

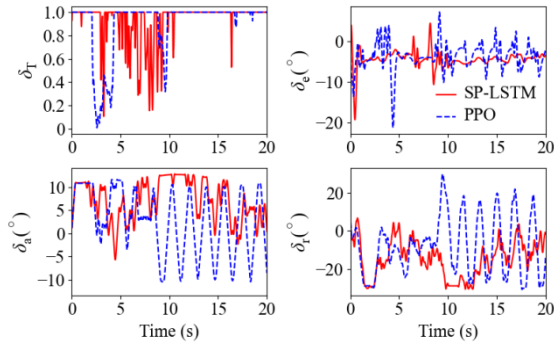
Model	Missile kill	Cannon kill	Enemy out of bounds	Total lose	Total win	Draw	Relative winning rate (%)
LSTM vs Base	193	2	88	17	283	0	94.3
LSTM vs ICM	37	0	72	103	109	88	51.4
LSTM vs PPO	92	0	168	2	260	38	99.2
ICM vs Base	226	1	42	22	269	9	92.4

To illustrate the decision-making process of the agent, we provide the action curves for the final optimized SP model, the fixed strategy model, the PPO algorithm model, and the basic SP model. In Fig. 19(a), the blue aircraft maintains a constant level flight, while the red aircraft executes a series of tactical maneuvers. The red one initially deflects the elevator upward. Concurrently, the throttle is advanced to increase acceleration. The red aircraft then applies positive deflections to the ailerons and negative deflections to the rudder, facilitating a high-g left turn. Upon aligning with the enemy's tail, the rudder deflection is adjusted to attempt a target lock on the adversary aircraft. In Fig. 19(b), it is observed that the PPO model exhibits significant overfitting when confronted with a highly capable opponent. This overfitting manifests as oscillations in the aileron and rudder deflections, which are indicative of the model's instability. This phenomenon is further demonstrated in Fig. 12(b), where action redundancy becomes apparent, reflecting the model's compromised decision-making efficiency. Comparing the effects of the optimized SP method and the basic SP algorithm is challenging based on Fig. 19(c) alone, as both methods exhibit long-term, double-loop engagements in their trajectories. However, the algorithm proposed in this paper demonstrates the capability to make more rational decisions across a broader range of scenarios, ultimately leading to a

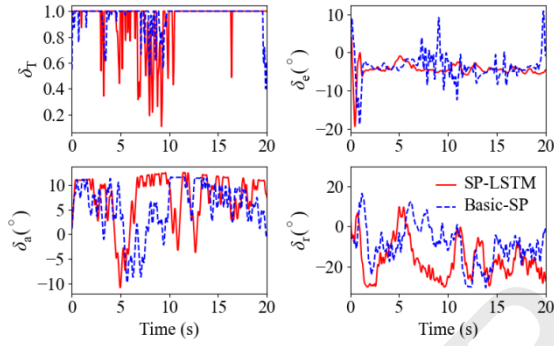
higher winning rate.



(a) SP-LSTM vs Heuristic



(b) SP-LSTM vs PPO



(c) SP-LSTM vs Basic SP

Fig. 19 Action curves in confrontation.

It is important to note that the implementation of the 6-DOF dynamic model, continuous action space, and end-to-end control results in more frequent adjustments to the agent's control surfaces. Despite this, the flight trajectory does not exhibit high-frequency oscillations, and the control logic aligns with human operational patterns. With further optimization and refinement of the control mechanisms, this method is anticipated to achieve even better performance.

7.3. Human-agent confrontation

To comprehensively assess the model's capabilities and application value, the team conducted a human-agent confrontation test in August 2023. This evaluation utilized the visual platform's first-person perspective, hardware interaction, and data saving and loading functions.

We organized a competitive evaluation involving pilots (Level I) and students (Level II) against the optimized SP model in a close head-on starting scenario. To emphasize the performance of the proposed method (Model A), the Basic SP model (Model B), which was trained in our previous work, was included as a baseline for horizontal comparison.

The results indicate that Model A significantly outperforms Model B, achieving a winning rate exceeding 75%, as shown in Table 12. This superior performance can be attributed to Model B's inability to handle the flexible maneuvers facilitated by hardware input, leading to passive agent behavior. In contrast, Model A was rated by testers as highly aggressive, with its intelligent decision-making closely aligning with conventional maneuvers, thereby providing valuable insights. Additionally, the performance of Level I testers, who are the most skilled, demonstrated the ability to sustain prolonged engagements with the agent and secure a relatively high number of victories. This outcome supports the objectivity of the test and further validates the effectiveness of Model A.

Table 12 Model confrontation result.

test	Win	Draw	Lose	Total	Relative winning rate(%)
A vs I	59	4	12	75	83.1
A vs II	55	3	2	60	96.5
B vs I	2	2	26	30	7.1
B vs II	5	3	22	30	18.5

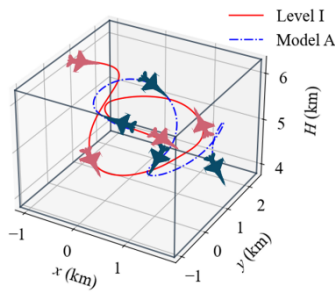
Given the prolonged nature and complexity of the confrontations, relying solely on static images in the visual platform does not effectively convey the trajectory details. Therefore, this section selects five representative trajectories—covering victories, defeats, and draws—from the confrontation data between Model A and Level I pilots. These trajectories are analyzed and illustrated using three-dimensional graphics to provide a clearer and more comprehensive understanding. Dynamic video and rendering effects further enhance the visualization of the air combat process. Due to space constraints, a trajectory demonstration under the visual platform is provided in the Appendix A.

Fig. 20(a) illustrates the trajectory of a pilot's victory. Initially, the pilot chooses to climb, leading both aircraft to engage in simultaneous turns. The maneuver transitions into a vertical plane engagement, with the red team executing a dive followed by a climb to complete an oblique plane turn. This maneuver successfully positions the red aircraft behind the enemy, resulting in a missile lock-on and subsequent kill. In Fig. 20(b), the

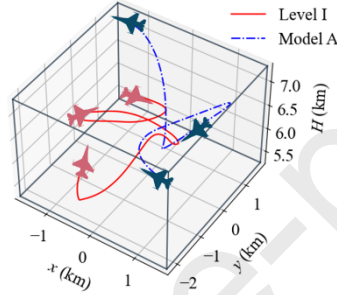
pilot employs vertical maneuvers to ascend from a low altitude, culminating in an upward missile attack against the agent. In the model training, we placed greater emphasis on angular advantage, and the SP training method inherently guides the strategy towards optimizing symmetric maneuvers, such as the dual-ring maneuver. As a result, the model is stronger in making decisions for plane-based maneuvers, while its response to energy maneuvers outside of the plane is weaker. When the pilot adopts reasonable vertical maneuvers, victory becomes more achievable.

In Fig. 20(c), the maneuvers of both sides are

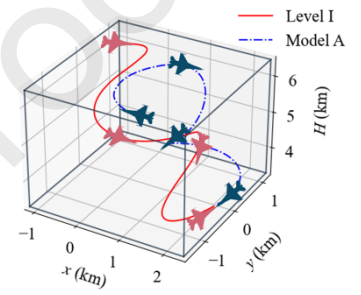
relatively symmetrical. After a period of stalemate, the close proximity of the two aircraft leads to a collision. Fig. 20(d) depicts a scenario where both aircraft initially maintain altitude and engage in a double-loop turn. Subsequently, the pilot descends and attempts to lock onto the enemy aircraft. However, due to excessive speed, the pilot is unable to secure the lock and is ultimately shot down by the enemy while advancing aggressively. Fig. 20(e) illustrates a typical double-loop tactic in close air combat. After a prolonged engagement, the pilot gains a positional advantage. Despite this, when attempting to lock onto the enemy, the opponent climbs to evade, and the red aircraft's low altitude prevents a successful strike, resulting in a missed opportunity.



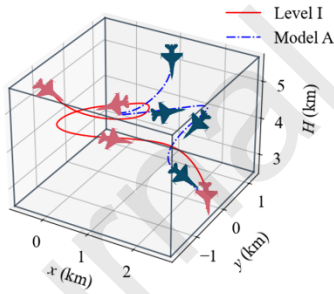
(a) Pilot victory 1



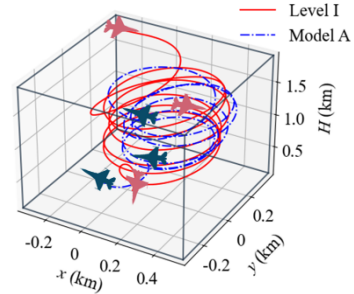
(b) Pilot victory 2



(c) Draw



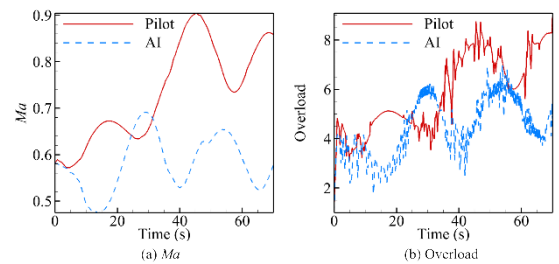
(d) Agent victory 1



(e) Agent victory 2

Fig. 20 Human-Agent Confrontation typical trajectories.

Through data storage and playback, it was observed that the agent demonstrates improved control over the fighter's speed near the cornering speed. The agent also manages maneuvering overloads effectively, as shown in Fig. 21, thereby balancing a smaller turning radius with a higher turn rate. This capability provides a robust explanation for the agent's high winning rate.



(a) Ma

(b) Overload

Fig. 21 State curves in a confrontation process.

To illustrate the relationship between simulation data and trajectory curves, we present in Fig. 22 the variations in altitude, Mach number, roll angle, and yaw angle corresponding to the agent's winning trajectory 2. The periodic changes in yaw angle and sustained large roll angles, accompanied by a decrease in altitude, align with the typical maneuvering trajectory of a dual-ring combat scenario at each moment. We can observe that during the dual-ring maneuver, both sides continuously attempted to enter the enemy's rear attack zone for a lock-on attack. However, the pilot struggled to control speed effectively, resulting in a larger turning radius and angular rate, thus requiring higher G-forces to avoid being locked by the enemy.

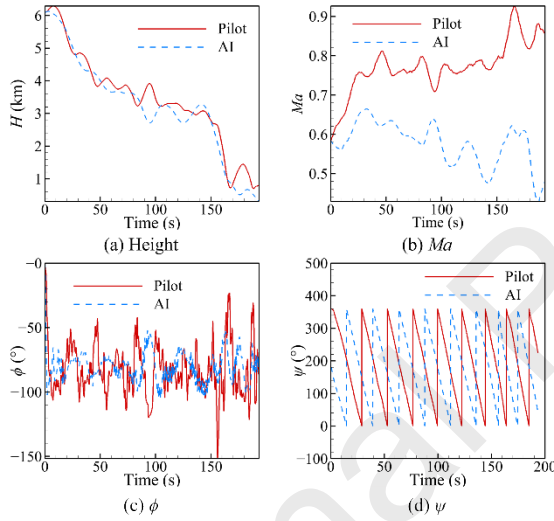


Fig. 22 State variables corresponding to the agent victory 2.

From the azimuth angle, we can see that the AI maintained an approximately 30° advantage throughout the engagement. As the altitude continued to decrease, the AI assessed the training scenario and determined that continuing the dual-ring maneuver would significantly increase the risk of ground collision. Therefore, it adjusted its strategy by making a small-angle turn and reducing its descent rate. After observing the Blue side's actions, the Red side also attempted a maneuver, but due to excessive speed, it struggled to execute a quick maneuver, ultimately failing and crashing into the ground. Furthermore, through the dynamic visualization of the visual system combined with the data playback feature, the relationship between simulation data and motion becomes clearer over synchronized time progression. In air combat research, simulation data

curves are often not intuitive, leading to a reliance on trajectory representations for explanation. However, the platform developed in this paper effectively integrates these two aspects. As the realism of the controlled object model continues to improve, the value of the simulation data is expected to increase significantly.

8. Conclusions

To meet the demands of military applications, we developed a six-degree-of-freedom fighter dynamics model using real aerodynamic data. This model features an optimized algorithmic logic and network architecture, including an LSTM layer, which enhances its ability to handle sequential decision-making. By integrating an improved SP method and conducting decision trajectory analysis and confrontation testing within a simulation visualization platform, we ensure comprehensive training and evaluation. The trajectory analysis demonstrates that our approach effectively combines expert strategic information with innovative reward structures, significantly improving the model's decision-making generalization and enabling it to adapt to diverse strategies from various opponents in random scenarios. In human-agent confrontation tests, the intelligent agent exhibited a decisive advantage over experienced pilots, achieving over 75% win rate while maintaining rational maneuvering. This validates both the model's performance and its practical application value, ultimately providing strong support for future implementation in real-world scenarios.

From an application perspective, this paper utilizes the F-16 model, which is widely used in flight simulation software and encompasses real aerodynamic and flight control systems. First, the algorithm can integrate expert strategies with flight data, continuously optimizing itself. The most direct application, given current hardware conditions, is in ground-based simulation training environments. By embedding this AI model into a simulation environment, it can significantly enhance pilot training efficiency, reduce the human and material resources required for training, and be used for early pilot selection and air combat experiences, allowing more people to understand air combat. Second, as the model's capabilities and architecture are further enhanced, it can be used for countermeasure research. In specific scenarios, the AI can output particular strategies, attempting to explore new maneuvering actions. Third, after validation and matching, the model can be deployed on flight control computers to provide pilots with operational cues within certain permissions, reducing pilot workload, or it can be applied to drone swarm operations, greatly enhancing drone combat capabilities. While there may be minor discrepancies

from actual fighter control, the model sufficiently meets flight simulation requirements and can serve as an opponent for simulation training. Future work will focus on improving the model's authenticity and reliability, standardizing fighter decision-making actions, and exploring collaborative scenarios involving multiple agents.

Acknowledgement

This study was co-supported by the National Natural Science Foundation of China (No. 91852115).

References

- [1] Sun ZX, Yang SQ, Piao HY, et al. A survey of air combat artificial intelligence. *Acta Aeronaut Astronaut Sin* 2021;**42**(8):525799 [Chinese].
- [2] Sun C. Development trend of future fighter: A review of evolution of winning mechanism in air combat. *Acta Aeronaut Astronaut Sin* 2021;**42**(8):525826 [Chinese].
- [3] Fan HT, Yan J. Evolution and development trend of air combat system. *Acta Aeronaut Astronaut Sin* 2022;**43**(10):527397 [Chinese].
4. ACE. Air Combat Evolution [Internet]. Arlington: Defense Advanced Research Projects Agency; c1996-02[updated 2025 Jan 09; cited 2025 Jan 09]. Available from: <https://www.darpa.mil/program/air-combat-evolution/>.
- [5] Yang W. Development of future fighters. *Acta Aeronaut Astronaut Sin* 2020;**41**(6): 524377 [Chinese].
6. Stillion J. Trends in air-to-air combat: Implications for future air superiority. *Center for strategic and budgetary assessments*. 2015.
- [7] Shi ZK. Challenge of control theory in the presence of high performance aircraft development. *Acta Aeronaut Astronaut Sin* 2015;**36**(8):2717–34 [Chinese].
- [8] Liu Q, Li XY, Yuan SH, et al. Decision-making technology for autonomous vehicles: Learning-based methods, applications and future outlook. *2021 IEEE international intelligent transportation systems conference (ITSC)*. Piscataway: IEEE Press; 2021.
- [9] Isaacs R. *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. 2nd ed. Mineola, N.Y.: Dover Publications; 1999.
- [10] Baron S. Differential games and manual control. *IEEE Trans Hum Factors Electron* 1966; **7**(4):133–7.
- [11] Park H, Lee BY, Tahk MJ, et al. Differential game based air combat maneuver generation using scoring function matrix. *Int J Aeronaut Space Sci* 2016;**17**(2):204–13.
- [12] Xu GY, Wei SN, Zhang HM. Application of situation function in air combat differential games. *2017 36th Chinese control conference (CCC)*. Piscataway: IEEE Press; 2017.
- [13] Gao WN, Yang Z, Huang JC, et al. Design of dynamic tactical control range in air combat based on three-party game. *2023 35th Chinese control and decision conference (CCDC)*. Piscataway: IEEE Press; 2023.
14. Burgin GH, Fogel LJ, Phelps JP. An adaptive maneuvering logic computer program for the simulation of one-on-one air-to-air combat. Volume 1: General description. Washington, D.C.: NASA; 1975. Report No.: 75N30817.
- [15] Akabari S, Menhaj MB, Nikraves SK. Fuzzy modeling of offensive maneuvers in an air-to-air combat. *Computational intelligence, theory and applications*. Berlin, Heidelberg: Springer; 2005. p. 171–84.
- [16] Ernest N, Carroll D. Genetic fuzzy based artificial intelligence for unmanned combat aerial vehicle control in simulated air combat missions. *J Def Manag* 2016;**6**(1): 2167-0374.
- [17] İşci H, Günel GÖ. Fuzzy logic based air-to-air combat algorithm for unmanned air vehicles. *Int J Dyn Contr* 2022;**10**(1):230–42.

- [18] Silver D, Huang A, Maddison CJ, et al. Mastering the game of Go with deep neural networks and tree search. *Nature* 2016;**529**(7587):484–9.
- [19] Vinyals O, Babuschkin I, Czarnecki WM, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 2019;**575**(7782):350–4.
- [20] Kaufmann E, Bauersfeld L, Loquercio A, et al. Champion-level drone racing using deep reinforcement learning. *Nature* 2023;**620**(7976):982–7.
- [21] Pope AP, Ide JS, Mićović D, et al. Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. *IEEE Trans Artif Intell* 2023;**4**(6):1371–85.
- [22] Liu P, Ma YF. A deep reinforcement learning based intelligent decision method for UCAV air combat. Modeling, Design and Simulation of Systems. Singapore: Springer; 2017; p. 274–86.
- [23] Ma XT, Xia L, Zhao QC. Air-combat strategy using deep Q-learning. *2018 Chinese automation congress (CAC)*. Piscataway: IEEE Press; 2018.
- [24] Yang QM, Zhang JD, Shi GQ, et al. Maneuver decision of UAV in short-range air combat based on deep reinforcement learning. *IEEE Access* 2019;**8**:363–78.
- [25] Hu DY, Yang RN, Zuo JL, et al. Application of deep reinforcement learning in maneuver planning of beyond-visual-range air combat. *IEEE Access* 2021;**9**:32282–97.
- [26] Kurniawan B, Vamplew P, Papasimeon M, et al. An empirical study of reward structures for actor-critic reinforcement learning in air combat manoeuvring simulation. *AI 2019: Advances in artificial intelligence*. Cham: Springer International Publishing; 2019. p. 54–65.
- [27] Zhang HP, Wei YJ, Zhou H, et al. Maneuver decision-making for autonomous air combat based on FRE-PPO. *Appl Sci* 2022;**12**(20):10230.
- [28] Wang Y, Zhang XW, Zhou R, et al. Research on UCAV maneuvering decision method based on heuristic reinforcement learning. *Comput Intell Neurosci* 2022;**2022**(1):1477078.
- [29] Sun ZX, Piao HY, Yang Z, et al. Multi-agent hierarchical policy gradient for Air Combat Tactics emergence via self-play. *Eng Appl Artif Intell* 2021;**98**:104112.
- [30] Li LT, Zhou ZM, Chai JJ, et al. Learning continuous 3-DoF air-to-air close-in combat strategy using proximal policy optimization. *2022 IEEE conference on games (CoG)*. Piscataway: IEEE Press; 2022.
- [31] Li B, Huang JY, Bai SX, et al. Autonomous air combat decision-making of UAV based on parallel self-play reinforcement learning. *CAAI Trans Intell Technol* 2023;**8**(1):64–81.
- [32] Jiang FL, Xu MQ, Li YQ, et al. Short-range air combat maneuver decision of UAV swarm based on multi-agent Transformer introducing virtual objects. *Eng Appl Artif Intell* 2023;**123**:106358.
33. Shan S, Yang M, Zhang W, et al. Continuous decision-making method for autonomous air combat. *Advances in Aeronautical Science and Engineering* 2022;**13**(5):47-58[Chinese].
34. Shan S, Zhang W. Air combat intelligent decision-making method based on self-play and deep reinforcement learning. *Acta Aeronautica et Astronautica Sinica*, 2024; 45(4): 328723 [Chinese].
- [35] Nguyen LT, Ogburn ME, Gilbert WP, et al. Simulator study of stall/post-stall characteristics of a fighter airplane with relaxed longitudinal static stability. 1979. Washington, D.C.: NASA; Report No: NASA-TP-1538.
- [36] Berndt J. JSBSim: An open source flight dynamics model in C++. *AIAA modeling and simulation technologies conference and exhibit*. Reston: AIAA; 2004.
- [37] Yu Y, Si XS, Hu CH, et al. A review of recurrent neural networks: LSTM cells and network architectures. *Neural Comput* 2019;**31**(7):1235–70.

- [38] Juliani A, Berges VP, Teng E, et al. Unity: a general platform for intelligent agents. arXiv preprint: 1809.02627; 2018.
- [39] Osinga F. ‘Getting’ a discourse on winning and losing: a primer on boyd’s ‘theory of intellectual evolution’. *Contemp Secur Policy* 2013;**34**(3):603–24.
- [40] Sutton RS, Barto AG. *Reinforcement learning: An introduction*. 2nd ed. MIT press; 2018.
- [41] Mnih V, Kavukcuoglu K, Silver D, et al. Playing atari with deep reinforcement learning. arXiv preprint: 1312.5602; 2013.
- [42] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms. arXiv preprint: 1707.06347; 2017.
- [43] Lillicrap TP, Hunt JJ, Pritzel A, et al. Continuous control with deep reinforcement learning. arXiv preprint: 1509.02971; 2015.
44. Haarnoja T, Zhou A, Abbeel P, et al. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International conference on machine learning*. 2018.
- [45] Sutton RS, McAllester D, Singh S, et al. Policy gradient methods for reinforcement learning with function approximation. *Proceedings of the 13th international conference on neural information processing systems*. New York: ACM; 1999.
- [46] Hernandez D, Denamganai K, Gao Y, et al. A generalized framework for self-play training. *2019 IEEE conference on games (CoG)*. Piscataway: IEEE Press; 2019.
- [47] Bai Y, Jin C, Bai Y, et al. Provable self-play algorithms for competitive reinforcement learning. *Proceedings of the 37th international conference on machine learning*. New York: ACM; 2020.
48. Russell RS. Non-linear F-16 simulation using Simulink and Matlab. Minnesota: University of Minnesota; 2003.
- [49] Zhang YX, Martinez-Rau LS, Oelmann B, et al. Enabling autonomous structural inspections with tiny machine learning on UAVs. *2024 IEEE sensors applications symposium (SAS)*. Piscataway: IEEE Press; 2024.
- [50] Sever M, Ogut S. A performance study depending on execution times of various frameworks in machine learning inference. *2021 15th Turkish national software engineering symposium (UYMS)*. Piscataway: IEEE Press; 2021.

Appendix A

Screenshots in chronological order from Figs.A1(a) to (h). The red side is controlled by the pilot and the blue side is controlled by the agent. This trajectory shows the beginning of the head-on situation between the two sides. The pilot first chooses the double-loop maneuver to try to gain the angle advantage, then turns to the vertical maneuver to pull up after falling into a disadvantage, and finally successfully dives to lock the blue side.

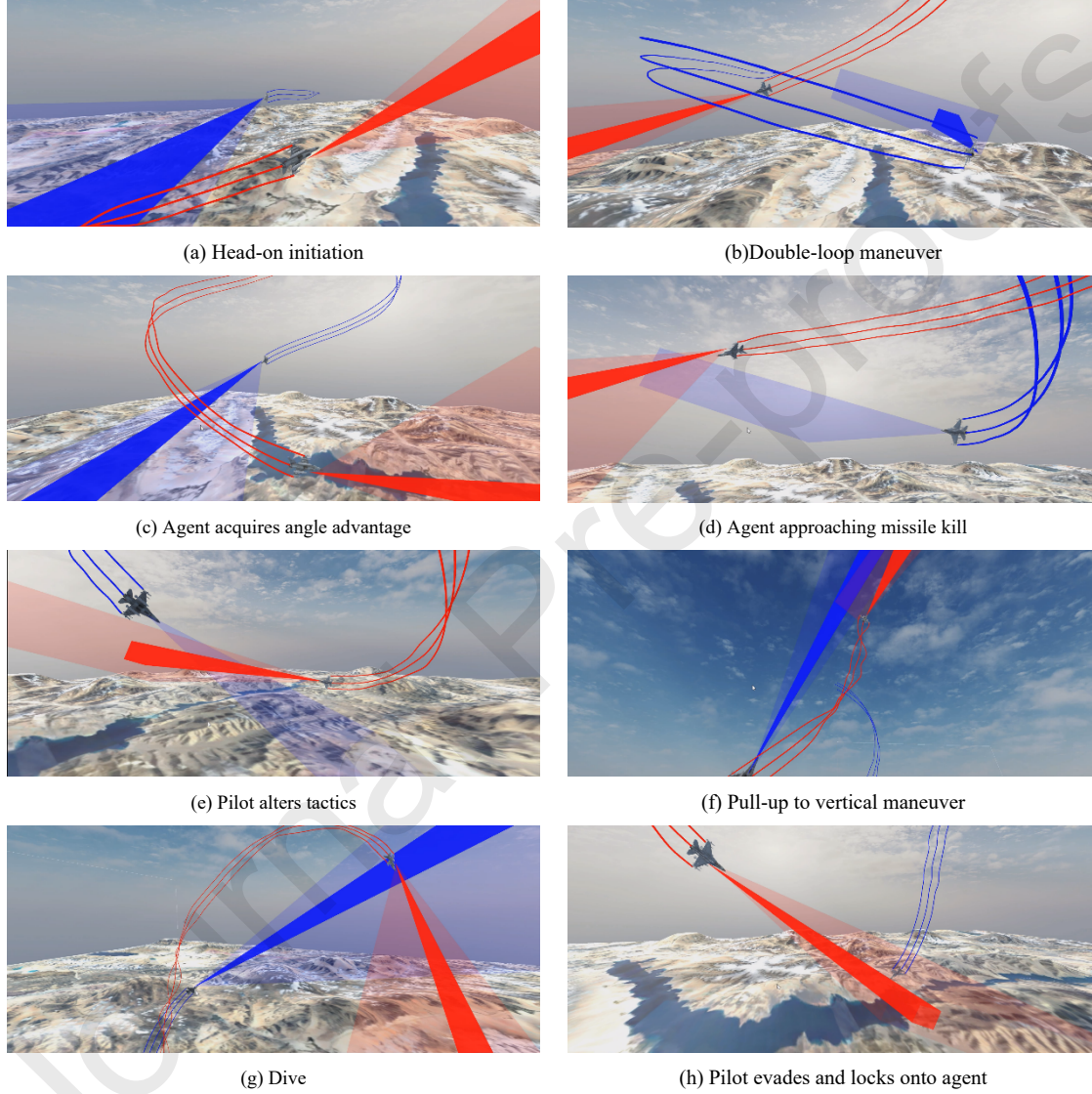


Fig.A1 Time series screenshots of a pilot's winning trajectory.

Declaration of interests

☒ The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐ The author is an Editorial Board Member/Editor-in-Chief/Associate Editor/Guest Editor for *[Journal name]* and was not involved in the editorial review or the decision to publish this article.

☐ The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

--